BAN HỌC TẬP CÔNG NGHỆ PHẦN MỀM

TRAINING CUỐI KỲ HỌC KỲ I NĂM HỌC 2024 – 2025

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

BAN HỌC TẬP

Khoa Công nghệ Phần mềm

Trường Đại học Công nghệ Thông tin

Đại học Quốc gia thành phố Hồ Chí Minh

CONTACT

bht.cnpm.uit@gmail.com

fb.com/bhtcnpm

fb.com/groups/bht.cnpm.uit

TEAM TIẾNG ANH

english.with.bht@gmail.com

creative.owl.se

english.with.bht

TRAINING

NHẬP MÔN LẬP TRÌNH

Thời gian: 9.30 thứ 6 ngày 03/01/2025

Địa điểm: Phòng B5.14

Trainers: Nguyễn Khả An – KTPM2024.1

Tiền Minh Dương – KTPM2023.1

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM

BAN HỌC TẬP

Sharing is learning

NỘI DUNG

I. Các kiểu dữ liệu và phép toán

II. Cấu trúc điều khiển

III. Mảng và Chuỗi

IV. Lập trình hàm

V. Đệ quy

VI. Kiểu cấu trúc

VII. Con trỏ

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Chương 1

KIỂU DỮ LIỆU CƠ BẢN
PHÉP TOÁN

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. KIỂU DỮ LIỆU

1. SỐ NGUYÊN

Kiểu dữ liệu	Kích thước	Giá trị có thể lưu
short	2 byte	Số nguyên
unsigned short	2 byte	Số nguyên không âm
int	4 byte	Số nguyên
unsigned int	4 byte	Số nguyên không âm
long long	8 byte	Số nguyên
unsigned long long	8 byte	Số nguyên không âm

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. KIỂU DỮ LIỆU

1. SỐ NGUYÊN

- Từ kích thước byte, có thể tính ra số bit (1 byte bằng 8 bit). Từ đó bạn có thể tính như sau:
  - Số có dấu: Tốn 1 bit để lưu trữ dấu
  - Phạm vi lưu trữ từ **-2^k đến 2^(k – 1) - 1**
  - Số không dấu: Kiểu dữ liệu K bit sẽ có thể lưu trữ từ 0 tới 2^k – 1
- Số nguyên không thể lưu phần thập phân -> nó sẽ cắt đi phần thập phân khi ép kiểu
- Nếu lưu số từ **-2e9 đến 2e9** thì sử dụng **long long**

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. KIỂU DỮ LIỆU

2. SỐ THỰC

Kiểu dữ liệu	Kích thước	Độ chính xác
float	4 byte	6 - 9
double	8 byte	15 - 18
long double	16 byte	33 - 36

- Thông thường mình sẽ dùng **double** cho các bài toán cần lưu thập phân
- Làm tròn số thực với độ chính xác 2 chữ số: **round(x*100) / 100**
- **Định dạng kiểu xuất số thực**
  - Kiểu khoa học: cout<<scientific
  - setprecision(x): định dạng bao nhiêu chữ số sẽ xuất hiện (bao gồm cả thập phân và nguyên). Khi thêm fixed, nó sẽ chỉ tính sau phần thập phân

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. KIỂU DỮ LIỆU

3. BOOLEAN

- Là kiểu dữ liệu để lưu giá trị đúng (**true)** hoặc **sai (false)**
- **Có kích thước 1 byte**
- Các giá trị tương ứng
    - **0, '\0', "": false**
    - **Các số nguyên, kí tự... : true**

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. KIỂU DỮ LIỆU

4. KÍ TỰ

Kiểu dữ liệu	Kích thước	Phạm vi lưu trữ
char	1 byte	-128 -> 127
unsigned char	1 byte	0 - 255

- Thực tế, char cũng có thể lưu giá trị số, đó là mã ASCII tương ứng của mỗi kí tự
- Vì lí do đó, ta có thể ép kiểu số nguyên sang char
- Các kí tự: chữ cái, chữ số, kí tự đặc biệt,… đều được lưu trữ bằng **char**
- **Phải nằm trong cặp ' '**

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. BIẾN (VARIABLES)

1. BIẾN

- Biến dùng để lưu trữ kết quả tính toán cho chương trình
- **<u>Cú pháp khai báo:</u>**

  [KIEU DU LIEU] [TEN BIEN]
- Có thể khai báo nhiều biến cùng KDL trên 1 dòng, cách nhau bằng dấu ,

```cpp
#include <iostream>

using namespace std;

int main(){
    int a = 28;
    long long b = 282828282828282;
    float c = 3.8912;
    double d = 10.8912781;
    char kitu = '@';
    bool check = true;
    cout << "Gia tri cua a : " << a << endl;
    cout << "Gia tri cua b : " << b << endl;
    cout << "Gia tri cua c : " << c << endl;
    cout << "Gia tri cua d : " << d << endl;
    cout << "Gia tri cua kitu : " << kitu << endl;
    cout << "Gia tri cua check : " << check << endl;
}
```

Sharing is learning

II. BIẾN (VARIABLES)

2. MỘT SỐ LƯU Ý

- Nếu khai báo biến mà không khởi tạo giá trị thì biến đó sẽ lấy 1 giá trị bất kỳ đang được lưu trong ô nhớ (giá trị rác)
- **Cú pháp khởi tạo** **[TEN BIEN] = [GIA TRI]**

  [KIEU DU LIEU] [TEN_BIEN] = [GIA_TRI]
- Ép kiểu dữ liệu: (Type casting): **(kieu_du_lieu)[ten bien]**
- **VD**: double a = 4.5

  cout<<(int)a

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. BIẾN (VARIABLES)

3. QUI TẮC ĐẶT TÊN BIẾN

- Không bắt đầu bằng số
- Không chứa dấu cách hoặc các kí tự đặc biệt
- Không trùng với các keywords có sẵn
- Không đặt trùng tên trong cùng 1 scope
- Phân biệt hoa thường

Ngoài ra, tên biến phải thể hiện được ý nghĩa của biến đó và tránh quá dài

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. BIẾN (VARIABLES)

4. HẰNG SỐ

- Hằng số là biến có giá trị không đổi trong suốt quá trình thực hiện chương trình
- Khai báo:

const [KIEU DU LIEU] [TEN] = [GIA TRI]

VD: const int maxn = 1e9;

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. BIẾN (VARIABLES)

5. KEYWORDS

- Là từ khóa được định nghĩa sẵn trong C++
- Không được sử dụng nó như tên biến

Sharing is learning

III. CÁC PHÉP TOÁN

1. TOÁN TỬ GÁN (=)

- Dùng để gán giá trị cho biến trước khi thực hiện tính toán

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n = 28;
    cout << "Gia tri cua bien n la: "<< n <<endl;
    int m = n;
    cout <<"Gia tri cua bien m la: "<< m << endl;
    int p = m;
    cout <<"Gia tri cua bien p la: "<< p << endl;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CÁC PHÉP TOÁN

2. PHÉP TOÁN SỐ HỌC

	Ý nghĩa	Ví dụ
+	Cộng	int n = 100 + 200
-	Trừ	int n = 100 - 200
*	Nhân	int n = 100 * 200
/	Chia	int n = 100 / 200
%	Chia dư	int n = 100 % 200
++	Tăng lên 1	int n = 100++
--	Giảm đi 1	int n = 100--

Ta có thể kết hợp với toán tử gán

	Ví dụ	Ý nghĩa
+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

BAN HỌC TẬP
Sharing is learning

III. CÁC PHÉP TOÁN

2. PHÉP TOÁN SỐ HỌC

Lưu ý với

- Nếu toàn bộ toán hạng đều là kiểu int thì kết quả sẽ là số nguyên (bỏ phần thập phân)
- Nếu có 1 toán hạng là kiểu thực thì đáp án là kiểu thực

```
int main()
{
    cout<<6/5<<endl;
    cout<<6.0/5<<endl;
}
```

```
1
1.2
Process returned 0 (0x0)
Press any key to continue.
```

BAN HỌC TẬP
Sharing is learning

III. CÁC PHÉP TOÁN

2. PHÉP TOÁN SỐ HỌC

Lưu ý với ++i và i++

-**++i:** Tăng giá trị của i lên 1 và trả về giá trị mới đó

- **i++:** Trả về giá trị ban đầu của i, sau đó mới tăng i lên 1
- Với các lệnh phức tạp thì i++ sẽ cho ra kết quả lâu hơn ++i
- Tuy nhiên, trong nhiều trường hợp, nó có thể thay thế cho nhau

```
int a = 5;
int x = a++;
int y = ++a;
cout << x << ' ' << y;
```

```
5 7
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CÁC PHÉP TOÁN

3. TOÁN TỬ SO SÁNH

Dùng để so sánh điều kiện. Trả về đúng hoặc sai khi so sánh 2 điều kiện với nhau

	Ý nghĩa	Cụ thể
>	Lớn hơn	a > b
>=	Lớn hơn hoặc bằng	a >= b
<	Nhỏ hơn	a <= b
<=	Nhỏ hơn hoặc bằng	a <= b
==	Bằng	a == b
!=	Khác	a != b

```
int main()
{
    cout << boolalpha << (5 < 6) << endl;
    cout << boolalpha << (5 == 6) << endl;
    cout << boolalpha << (5 < 6) << endl;
}
```

```
false
false
true
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CÁC PHÉP TOÁN

4. TOÁN TỬ LOGIC

Dùng để kết hợp nhiều toán hạng trong biểu thức điều kiện. Phần này cần xem lại 3 cổng logic **AND, OR** và **NOT**

Toán hạng	Ý nghĩa	Cụ thể
&&	AND	Cho giá trị đúng khi mọi toán hạng có giá trị đúng, sai trong các trường hợp còn lại
||	OR	Cho giá trị sai khi mọi toán hạng có giá trị sai, đúng trong các trường hợp còn lại
!	NOT	Phủ định lại

BAN HỌC TẬP
Sharing is learning

Chương 2

CẤU TRÚC ĐIỀU KHIỂN

ĐOÀN KHOA
ÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. CẤU TRÚC RẼ NHÁNH

1. CÂU LỆNH IF – ELSE

- Câu lệnh if – else giúp ta kiểm tra các điều kiện cho trước và thực hiện lệnh khi điều kiện được thỏa mãn
- VD: Nếu **n chia hết cho 2** in ra **n là số chẵn**

```
if [CONDITION]
{
        //codes
}
else (có thể có hoặc không)
{
        //codes
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. CẤU TRÚC RẼ NHÁNH

2. CÂU LỆNH ĐẦY ĐỦ

– Trong trường hợp có nhiều điều kiện rẽ nhánh. Ta có thể dùng cấu trúc **if – else if – else** để kiểm tra nhiều điều kiện trong 1 lệnh

```
if [CONDITION]
{
        //codes
}
else if
{
        //codes
}
else (có thể có hoặc không)
{
//codes
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. CẤU TRÚC RẼ NHÁNH

3. SWITCH – CASE

```
1. switch(<biểu_thức>)
2. {
3.     case <giá trị thứ 1>: <khối lệnh>;
4.             break;
5.     case <giá trị thứ 2>: <khối lệnh>;
6.             break;
7.     . . .
8.     default: <khối lệnh>;
9. }
```

Sharing is learning

I. CẤU TRÚC RẼ NHÁNH

3. SWITCH – CASE

- Thực hiện khối lệnh trong case tương ứng với kết quả của biểu thức trong switch
- Biểu thức phải có kết quả là giá trị nguyên
- **Default:** mặc định khi không có case nào được chọn
- Nếu không có lệnh break thì nó thực hiện tất cả case nằm ngay phía sau case được chọn

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

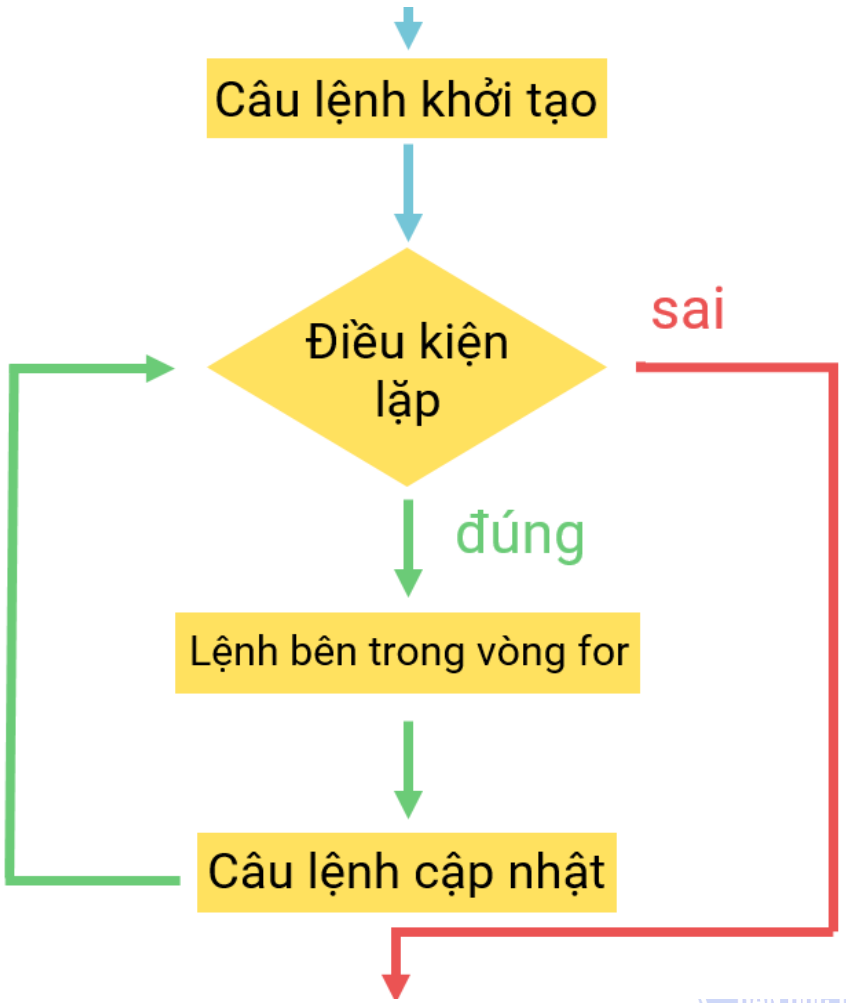
II. VÒNG LẶP

1. FOR

Vòng lặp for giúp thực hiện 1 câu lệnh nhiều lần theo ý bạn muốn

for(Lệnh khởi tạo; Điều_kiện_lặp; cập nhật)

- Ta có thể bỏ qua lệnh khởi tạo, đk lặp, cập nhật nhưng phải chừa ;



Sharing is learning

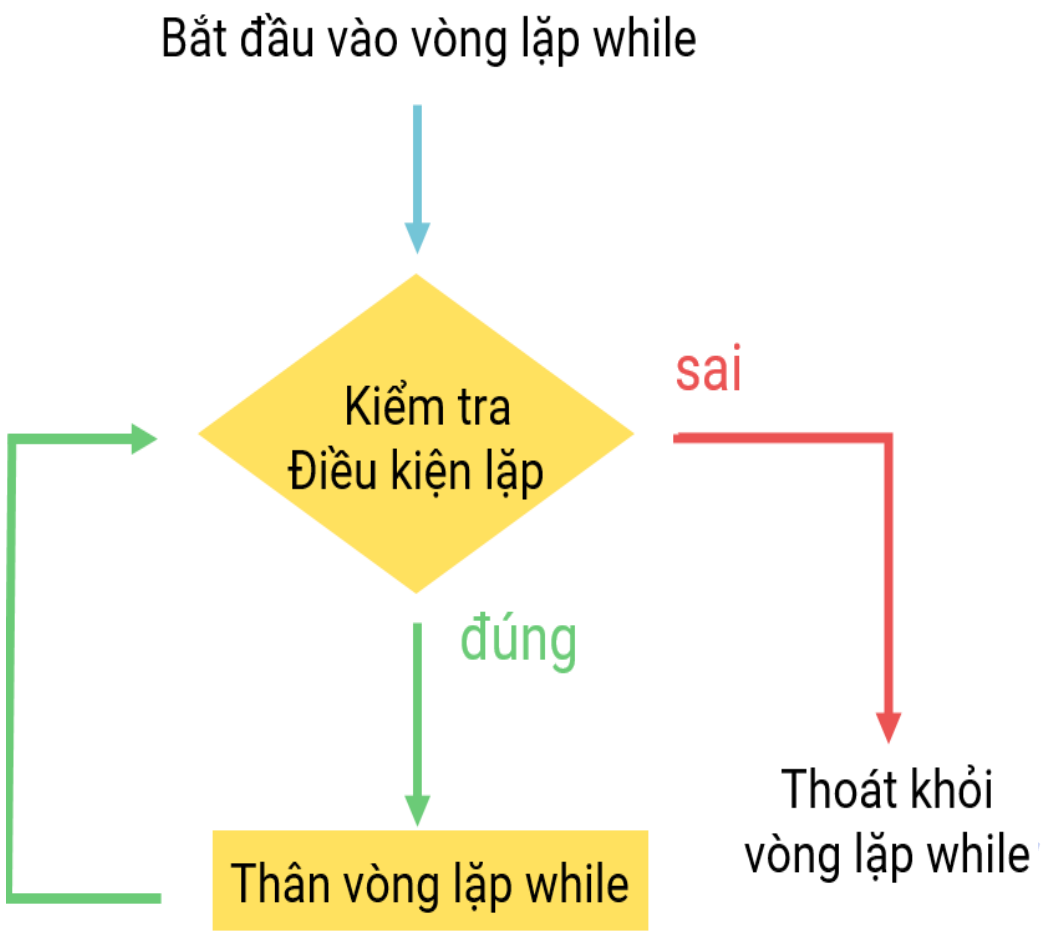
II. VÒNG LẶP

2. WHILE

Kiểm tra điều kiện và thực hiện câu lệnh khi đk còn đúng

```
while(condition) {
    //code
}
```

Nếu condition là 1 biểu thức luôn đúng thì ta có vòng lặp vô hạn



Sharing is learning

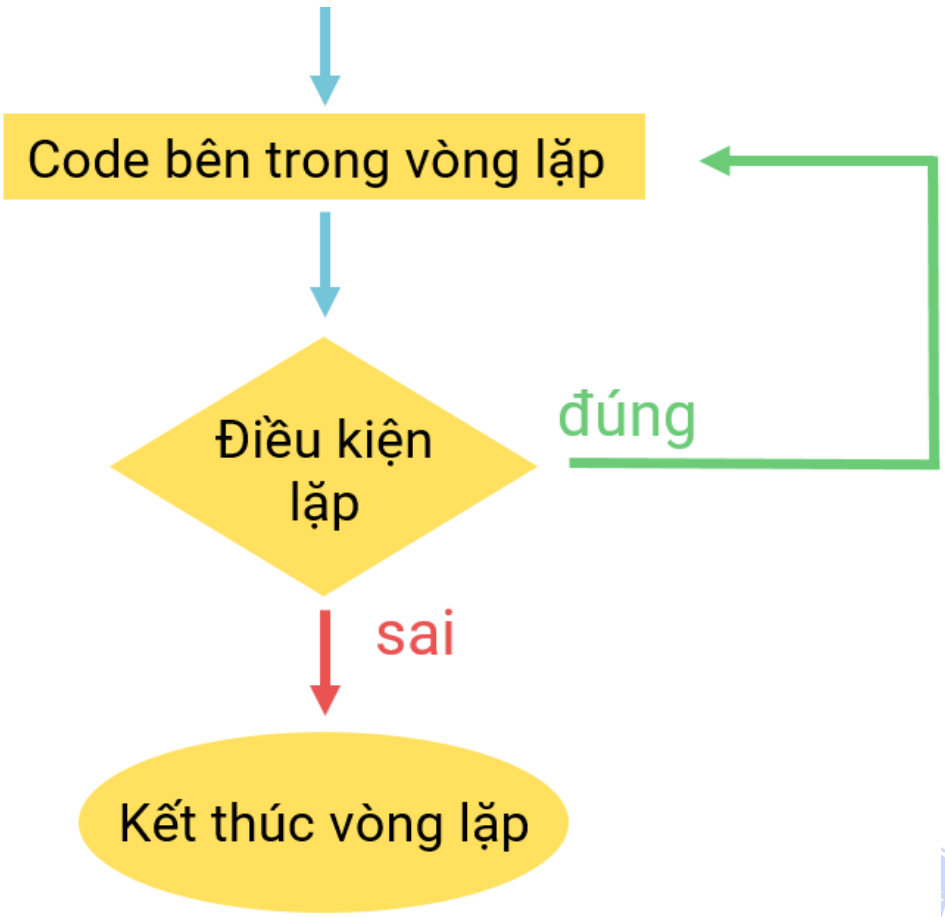
II. VÒNG LẶP

3. DO - WHILE

Thực hiện câu lệnh trước rồi kiểm tra điều kiện sau

```
do{
   //Code
} while(condition);
```

- Vòng do-while luôn thưc hiện code ít nhất 1 lần



BAN HỌC TẬP
Sharing is learning

II. VÒNG LẶP

4. BREAK

- break sử dụng khi muốn dừng vòng lặp. Các câu lệnh bên dưới break sẽ không được thực thi
- Có ý nghĩa với vòng lặp gần nó nhất
- Thông thường, break được dùng khi đã tìm ra được kết quả bài toán

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Lệnh break kết thúc vòng lặp khi người dùng nhập số >= 0

```cpp
#include <iostream>
using namespace std;
int main()
{
    int n;
    while (true)
    {
        cout<<"Nhap n: \n";
        cin >> n;
        if (n >= 0) break;
        else cout<<"Nhap lai \n";
    }
}
```

Sharing is learning

II. VÒNG LẶP

5. CONTINUE

- continue sẽ bỏ qua các câu lệnh bên dưới nó và tiến tới vòng lặp mới ngay
- Khi có các vòng lặp lồng nhau, continue có ý nghĩa với vòng lặp gần nó nhất

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. VÒNG LẶP

5. CONTINUE

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int main()
4 {
5     for (int i = 0; i < 12; ++i) {
6         if (i % 2 == 0) continue;
7         cout << i << endl;
8     }
9 }
```

Sharing is learning

Chương 3

MẢNG - CHUỖI

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. MẢNG

1. TỔNG QUAN

Mảng là 1 dãy các phần tử lưu cạnh nhau trong bộ nhớ có cùng kiểu dữ liệu

- Dễ cài đặt, truy cập phần tử
- Kích thước cố định, khó mở rộng, thu hẹp, chèn, xóa,…

VD: int a[200]: mảng các số nguyên có 200 phần tử

char b[2000]: mảng các kí tự có 2000 phần tử

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. MẢNG

2. KHAI BÁO

Kieu_du_lieu ten_mang [so_phan_tu]

<u>Lưu ý:</u> Số phần tử phải được ghi cứng hoặc là 1 biến const

```
const int maxn = 1e5;
int a[maxn];
int b[1000];
```

Ta có thể khai báo sẵn phần tử

```
int a[5]  ={1 , 2 , 3 , 4 , 5};
```

BOÀN KHOA
BAN HỌC TẬP
Sharing is learning

I. MẢNG

3. TRUY CẬP PHẦN TỬ

Các phần tử trong mảng được đánh số từ 0 -> N – 1

Cú pháp truy cập: **ten_mang[chi_so]**

Lưu ý:

- Ta vẫn có thể truy cập các giá trị ngoài phạm vi của mảng nhưng nó sẽ gây lỗi bộ nhớ hoặc sai kết quả :D. Thói quen tốt là nên quản lí truy cập hiệu quả

BỘ MÔN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. MẢNG

4. DUYỆT MẢNG

Ta dùng vòng lặp for / while để duyệt qua các phần tử trong mảng bằng chỉ số

```
const int N = 5;
int a[N] = {1 , 2 , 3 , 4 , 5};
for (int i = 0; i < 5; ++i) cout << a[i];
```

```
for (int i = 4; i >= 0; --i) cout <<a[i]<<' ';
```

BOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. MẢNG

5. NHẬP - XUẤT MẢNG

Nhập mảng

```
int a[1000];
int n;
cin >> n;
for (int i = 0; i < n; ++i) cin >> a[i];
```

Xuất mảng

```
for (int i = 0; i < n; ++i) cout << a[i];
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. MẢNG

5. TRUYỀN MẢNG VÔ HÀM

Bạn có thể truyền mảng làm tham số cho hàm, thường chúng ta nên kèm theo 1 tham số n là kích thước sử dụng của mảng 😚

```
1 void greet(int a[], int n)
2 {
3     for (int i = 0; i < n; ++i) cout << a[i]
4 }
5 int main()
6 {
7     int a[] = {1 , 2 , 3 , 4 , 5};
8     greet(a,5);
9 }
```

Note: Truyền mảng là truyền tham chiếu

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. MẢNG 2 CHIỀU

1. TỔNG QUAN

Mảng 2 chiều tương tự như 1 bảng số. Mảng 2 chiều là 1 mảng gồm các mảng 1 chiều

- **Cú pháp**: **[kieu_du_lieu] [ten_mang] [so_hang][so_cot]**

VD: int a[20][20]: bảng 20 x 20 số nguyên

char b[2][10]: bảng 2 x 10 kí tự

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. MẢNG 2 CHIỀU

2. TRUY CẬP PHẦN TỬ

- Truy cập phần tử trong mảng 2 chiều qua chỉ số hàng trước, cột sau
- Trong mảng 2 chiều không được truy cập không hợp lệ

```
int a[105][105];
for (int i = 0; i < 100; ++i)
    for (int j = 0; j < 100; ++j) cin >> a[i][j];
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. MẢNG 2 CHIỀU

3. NHẬP XUẤT MẢNG 2 CHIỀU

```
int main()
{
    int m,n;
    cin >> m >> n;
    const int N = 1e3;
    int a[N][N];
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];
    for (int i = 0; i < m; ++i)
    {
        for (int j = 0; j < n; ++j) cout<<a[i][j]<<' ';
        cout<<endl;
    }
}
```

II. MẢNG 2 CHIỀU

4. MẢNG 2 CHIỀU VÀ HÀM

- Ta có thể truyền mảng 2 chiều vào hàm. Nhưng bạn phải khai báo số cột

```
void greet(int a[][10])
{
    for (int i = 0; i < 10; ++i)
        for (int j = 0; j < 10; ++j) cout << a[i][j];
}
int main()
{
    int a[30][30];
    greet(a);
}
```

III. CHUỖI

1. KHAI BÁO, KHỞI TẠO

Các kiểu khai báo chuỗi

char s[100];

char s[];

char *s;

0	1	2	3	4	5	6	7	8	9
'U'	'I'	'T'	'\0'	'\0'	'\0'	'\0'	'\0'	'\0'	'\0'

Khởi tạo sẵn giá trị cho chuỗi

char s1[100] = {'H' , 'e' , 'l' , 'l' , 'o' ,'w' , '\0'};

char st[] = "Hello world"; ***(Chương trình tự thêm kí tự null vào cuối chuỗi)***

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CHUỖI

2. NHẬP XUẤT, CHUỖI

a) Nhập chuỗi không khoảng trắng

- Ta dùng lệnh cin >> s;

- Câu lệnh cin sẽ đọc dữ liệu cho tới khi gặp kí tự khoảng trắng hoặc '\n'

```
1  int main()
2  {
3      char s[100];
4      cin >> s;
5      cout << s;
6  }
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CHUỖI

2. NHẬP XUẤT, CHUỖI

b) Nhập chuỗi có khoảng trắng

- cin.getline(s, MAX_N);

- Câu lệnh **cin.getline** sẽ đọc cho tới khi gặp dấu '\n' trong bộ nhớ đệm và **xóa nó đi**

```
int main()
{
    char s[100];
    cin.getline(s , 100);
    cout << s;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CHUỖI

2. NHẬP XUẤT, CHUỖI

c) Lỗi khi nhập xuất chuỗi

- Khi ta kết hợp giữa câu lệnh cin >> và cin.getline(). Nếu không khéo có thể gây ra kết quả không mong muốn

```
1 int main()
2 {
3     char st[100];
4     cin >> st;
5     char s[100];
6     cin.getline(s , 100);
7 }
```

Đầu tiên ta dùng lệnh **cin**, đọc cho tới khi ta nhấn Enter -> Nó lấy toàn bộ chuỗi trong bộ nhớ đệm lưu vào biến st, chừa \n' trong bộ nhớ. Lệnh **cin.getline()** đọc cho tới khi gặp kí tự '\n'. Lúc này nó thấy '\n' vẫn đang trong bộ nhớ đệm do lệnh cin -> nó không đọc nữa :D

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CHUỖI

2. NHẬP XUẤT, CHUỖI

c) Lỗi khi nhập xuất chuỗi

```
1  int main()
2  {
3      char st[100];
4      cin >> st;
5      cin.ignore();
6      char s[100];
7      cin.getline(s , 100);
8      cout<<s<<'\n'<<st;
9  }
```

Để khắc phục lỗi trên. Ta dùng **cin.ignore()** giúp bỏ qua 1 kí tự trong bộ nhớ đệm (hay nó sẽ bỏ qua '\n' để **cin.getline()** có thể bắt đầu đọc)

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CHUỖI

2. NHẬP XUẤT, CHUỖI

c) Lỗi khi nhập xuất chuỗi

```
1 int main()
2 {
3     char st[100];
4     cin.getline(st , 100);
5     cin.ignore();
6     char s[100];
7     cin.getline(s , 100);
8     cout<<s<<'\n'<<st;
9 }
```

cin.getline() đầu tiên lấy hết trong bộ nhớ đệm. **cin.ignore()** bỏ qua 1 kí tự trong bộ nhớ đệm nên lệnh thứ 2 bị mất đi 1 chữ đầu

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

III. CHUỖI

3. MỘT SỐ HÀM XỬ LÍ CHUỖI

Cú pháp	Ý nghĩa
strlen(s)	Trả về độ dài của chuỗi
strcpy(s1,s2)	Sao chép chuỗi s2 lưu vào chuỗi s1
strcmp(s1,s2)	So sánh chuỗi s1, s2. 0 : bằng nhau 1: s1 > s2 -1: s1 < s2
strcat(s1,s2)	Nối chuỗi s2 vào s1
strstr(s1,s2)	Trả về con trỏ trỏ tới vị trí đầu tiên của chuỗi con s2 trong s1
memset(s,<value>,sizeof(s))	Đặt tất cả kí tự của chuỗi về value

BAN HỌC TẬP
Sharing is learning

Chương 4

LẬP TRÌNH HÀM

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. PHẠM VI BIẾN

1. GLOBAL – LOCAL

- **Global:** biến được khai báo bên ngoài main() và tất cả các hàm. Có thể truy cập ở bất cứ đâu trong chương trình
- **Local:** Được khai báo trong main() hoặc các hàm khác, chỉ có thể truy cập trong hàm mà nó khai báo

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

I. PHẠM VI BIẾN

1. GLOBAL – LOCAL

```cpp
#include <bits/stdc++.h>
using namespace std;
int n = 12;
void greet() {
    cout << n << endl; //12
    int a = 98;
    cout << a << endl; //98
}
int main() {
    cout << n << endl; //12
    cout << a << endl; //Lỗi
}
```

Sharing is learning

II. LẬP TRÌNH HÀM

1. ĐỊNH NGHĨA

Hàm là một khối lệnh có nhiệm vụ thực hiện một chức năng nào đó

Code trở nên mạch lạc, dễ đọc

Dễ bảo trì khi cần thay đổi một chức năng nhỏ

Dễ debug khi gặp lỗi

Có khả năng tái sử dụng lại code

BAN HỌC TẬP
Sharing is learning

II. LẬP TRÌNH HÀM

2. CÚ PHÁP

```
Data_type function_name (parameters) {
……//code
}
```

- **Data type:** Kiểu trả về của hàm (int, long long, string) hoặc void (rỗng)
- **Function_name:** tên hàm
- **Parameters:** đầu vào của hàm

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LẬP TRÌNH HÀM

2. CÚ PHÁP

```
1 int tong(int a, int b, int c)
2 {
3     int sum = a + b + c;
4     return sum;
5 }
```

```
1 void greet()
2 {
3     cout << "hello world";
4     return;
5 }
```

BAN HỌC TẬP
Sharing is learning

II. LẬP TRÌNH HÀM

2. CÚ PHÁP

```
1 void greet();
2 int calc(int a, int b, int c);
```

- **Declaration function:** Cho compiler biết về tên hàm và cách định nghĩa hàm đó. Một lúc sau hàm đó sẽ được viết thêm phần body vào

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LẬP TRÌNH HÀM

3. LỜI GỌI HÀM

Sau khi khai báo hàm có thể sử dụng hàm đó bằng cách gõ tên hàm và truyền tham số (nếu cần).

Các câu lệnh trong hàm sẽ được thực hiện

```
#include <stdio.h>

void greet(){
    // code
}

int main(){
    .. ... ...
    greet(); // Function call
    .. ... ...
}
```

II. LẬP TRÌNH HÀM

3. LỜI GỌI HÀM

```cpp
#include <bits/stdc++.h>
using namespace std;
void greet(int a, int b)
{
    cout << a << ' ' << b;
}
int main()
{
    greet(4,3);
}
```

II. LẬP TRÌNH HÀM

4. ĐỐI SỐ VÀ THAM SỐ

- **Tham số (parameters):** Là tham số hình thức khi bạn xây dựng hàm
- **Đối số (arguements):** Là các giá trị bạn truyền vào cho hàm khi gọi hàm

```
1 void greet(int a, int b)
2 {
3     cout << a << ' ' << b;
4 }
```

```
1 greet(4,3);
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

II. LẬP TRÌNH HÀM

5. THAM TRỊ, THAM CHIẾU

Tham trị (pass by value): giá trị của đối số được gán cho tham số khi thực hiện hàm => Chỉ thao tác trên bản sao của đối số

```
0x61fe1c
0x61fdf0
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  void greet(int a)
4  {
5      cout << &a;
6  }
7  int main()
8  {
9      int a = 4;
10     cout<< &a << endl;
11     greet(a);
12 }
```

BAN HỌC TẬP
Sharing is learning

II. LẬP TRÌNH HÀM

5. THAM TRỊ, THAM CHIẾU

Tham chiếu(pass by ref): Thao tác trực tiếp trên chính biến đó. Biến đối số có thể thay đổi giá trị sau khi thực hiện hàm

```
0x61fe1c
0x61fe1c
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  void greet(int &a)
4  {
5      cout << &a;
6  }
7  int main()
8  {
9      int a = 4;
10     cout<< &a << endl;
11     greet(a);
12 }
```

Sharing is learning

Chương 5

ĐỆ QUY

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐỆ QUY

1. Khái niệm

– Một vấn đề mang tính đệ quy nếu như nó có thể được giải quyết thông qua kết quả của chính vấn đề đó nhưng với đầu vào đơn giản hơn.

– Ví dụ:

+ $n! = n.(n - 1)!$

+ $a^n = a.a^{n-1}$

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐỆ QUY

1. Khái niệm

– Hàm đệ quy là hàm **gọi lại chính nó** trong thân hàm của nó.

– Ví dụ:

```
int fib(int n)
{
    if (n <= 1) // Điều kiện dừng
        return n;
    return fib(n - 1) + fib(n - 2); // Đệ quy
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐỆ QUY

2. Các loại đệ quy

– Đệ quy tuyến tính

```
int giaithua(int n)
{
    if (n == 0)
        return 1;
    return n * giaithua(n - 1);
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐỆ QUY

2. Các loại đệ quy

– Đệ quy nhị phân

```
int fib(int n)
{
    if (n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐỆ QUY

2. Các loại đệ quy

– Đệ quy hỗ tương

```
bool isEven(int n)
{
    if (n == 0)
        return true;
    return isOdd(n - 1);
}

bool isOdd(int n)
{
    if (n == 0)
        return false;
    return isEven(n - 1);
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐỆ QUY

2. Các loại đệ quy

– Đệ quy phi tuyến:

```
int xn(int n)
{
    if (n == 0)
        return 1;
    int res = 0;
    for (int i = 1; i <= n; i++)
        res = res + i * xn(n - i);
    return res;
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Chương 6

STRUCT

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

STRUCT

1. Khái niệm

– Kiểu cấu trúc (hay struct) là một kiểu dữ liệu do người dùng tự định nghĩa, gồm tập hợp các thuộc tính liên quan tới một đối tượng.

– Ví dụ:

+ Đối tượng phân số có 2 thuộc tính là tử số và mẫu số

+ Đối tượng sinh viên có thể có các thuộc tính là tên, tuổi, MSSV, lớp, …

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

STRUCT

2. Khai báo struct

struct <tên struct>

{

<kiểu dữ liệu 1> <thuộc tính 1>;

<kiểu dữ liệu 2> <thuộc tính 2>;

...

};

```
1  struct SinhVien
2  {
3      string Ten;
4      int Tuoi;
5      string Lop;
6      float Diem;
7  };
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

STRUCT

3. Kích thước của struct

- Kích thước của struct trong C++ tùy thuộc vào các phần tử tạo nên nó.
- Kích thước của struct thường lớn hơn so với tổng kích thước các phần tử.
- Có thể dùng hàm `sizeof()` để kiểm tra kích thước của struct.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Chương 7

CON TRỎ

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CON TRỎ

1. Vùng nhớ

– RAM chứa rất nhiều ô nhớ với mỗi ô có kích thước 1 byte.

– Mỗi ô nhớ được đại diện bằng 1 số thứ tự, và con số này được xem là địa chỉ ô nhớ đó.

– Địa chỉ ô nhớ được hiển thị dưới dạng thập lục phân.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CON TRỎ

2. Con trỏ

- Con trỏ là một biến lưu địa chỉ của một biến khác.
- Cách khai báo con trỏ:

<kiểu dữ liệu> *<tên>;

Ví dụ: `int *p;`

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

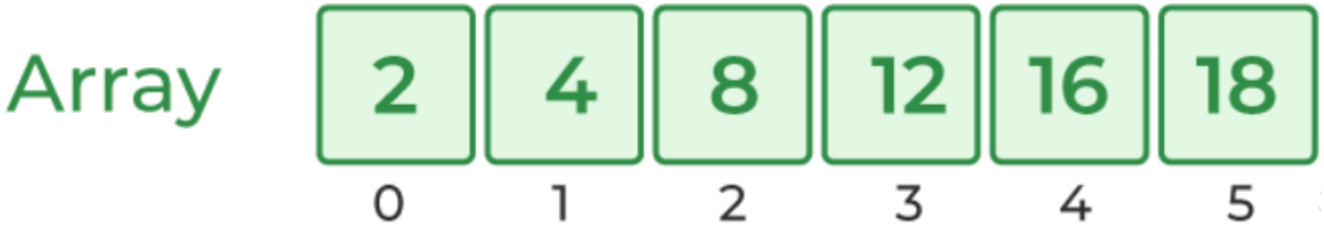
CON TRỎ

3. Địa chỉ của mảng

– Các phần tử của mảng được lưu ở những vị trí liên tiếp nhau trên bộ nhớ.

– Tên mảng là một con trỏ giữ địa chỉ của phần tử đầu tiên của mảng, cũng là địa chỉ của mảng.

– Địa chỉ của các phần tử trong mảng cách nhau một khoảng bằng kích thước của kiểu dữ liệu của phần tử trong mảng.



ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CON TRỎ

4. Các phép toán cơ bản với con trỏ

– Phép gán: được thực hiện như những phép gán thông thường, với vế phải là một địa chỉ của một biến có loại tương ứng với con trỏ (dùng toán tử & để lấy địa chỉ của một biến).

– Ví dụ:

```
int a = 5;
int *p = &a;
int **q = &p;
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CON TRỎ

4. Các phép toán cơ bản với con trỏ

– Phép truy xuất vùng nhớ: Dùng toán tử * để lấy giá trị mà vùng nhớ đang giữ.

– Ví dụ:

```
cout << *p << endl;
cout << *q << endl;
cout << **q << endl;
```

– Đối với con trỏ trỏ đến vùng nhớ của một biến kiểu cấu trúc, có thể dùng toán tử -> để truy cập trực tiếp thuộc tính.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CON TRỎ

4. Các phép toán cơ bản với con trỏ

– Có thể dùng các toán tử +, ++, +=, -, --, -= với con trỏ.

– Khi thực hiện cộng, trừ, các số nguyên ở các vế không trực tiếp là số nguyên đó mà là (số nguyên đó) * (kích thước kiểu dữ liệu của con trỏ tương ứng).

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CON TRỎ

4. Các phép toán cơ bản với con trỏ

– Ví dụ: Trong đoạn chương trình sau, giả sử &a = 0x61fe14

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int a = 5;
6      int *p = &a;
7      p += 4;
8      cout << &a << endl; // 0x61fe14
9      cout << p << endl; // 0x61fe24
10     cout << p - &a << endl; // 4
11     cout << p - 3 << endl; // 0x61fe18
12 }
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CON TRỎ

5. Hằng con trỏ, con trỏ hằng

- Hằng con trỏ là con trỏ mà không được thay đổi địa chỉ mà nó trỏ vào, được khai báo là <kiểu dữ liệu>* const <tên biến>;
- Ví dụ: `int* const p = &a;`
- Con trỏ hằng là con trỏ mà ta không được thay đổi giá trị của vùng nhớ mà con trỏ đó trỏ vào thông qua chính nó, được khai báo dạng const <kiểu dữ liệu>* <tên biến>;
- Ví dụ: `const int* p = &a;`

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

CON TRỎ

6. Toán tử new, delete

– Toán tử new được dùng để cấp phát một vùng nhớ trong C++ và trả về địa chỉ của vùng nhớ đó.

– VD: `int *p = new int(10);`

```
int *arr = new int[10];
```

– Để giải phóng vùng nhớ được cấp phát bởi toán tử new, ta dùng toán tử `delete` hoặc `delete[]` tương ứng.

– VD: Để giải phóng 2 vùng nhớ được cấp phát như trên, ta dùng: `delete p;` và `delete[] arr;` tương ứng.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
HỌC THUẬT

- Training
- Giải đáp
- Chia sẻ

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
TRUYỀN THÔNG

- Design ấn phẩm
- Viết content
- Chụp ảnh

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
TIẾNG ANH

- Instagram
- TikTok
- Dịch thuật
- Thi thử

BAN HỌC TẬP CÔNG NGHỆ PHẦN MỀM

TRAINING GIỮA KỲ HỌC KỲ II NĂM HỌC 2023 – 2024

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

HẾT

CẢM ƠN CÁC BẠN ĐÃ THEO DÕI
CHÚC CÁC BẠN CÓ KẾT QUẢ THI THẬT TỐT!

BAN HỌC TẬP

Khoa Công nghệ Phần mềm
Trường Đại học Công nghệ Thông tin
Đại học Quốc gia thành phố Hồ Chí Minh

CONTACT

bht.cnpm.uit@gmail.com
fb.com/bhtcnpm
fb.com/groups/bht.cnpm.uit

TEAM TIẾNG ANH

english.with.bht@gmail.com
creative.owl.se
english.with.bht